

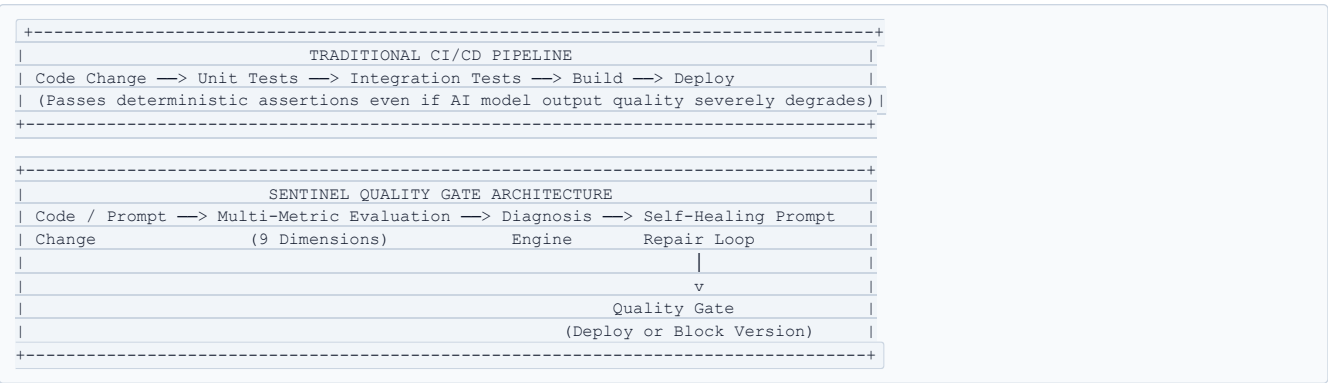
Author: Srishanth Reddy
Affiliation: Autonomous AI Systems Research Lab & SENTINEL Project
Date: September 2026
Document Version: 3.0.0

Large Language Models (LLMs) have transformed artificial intelligence applications across software engineering, enterprise search, and automated decision-making. However, the inherently non-deterministic nature of LLMs introduces severe vulnerabilities in production environments—including hallucination drift, context misalignment, instruction non-compliance, and silent prompt regression. Traditional Continuous Integration and Continuous Deployment (CI/CD) pipelines evaluate deterministic software assertions (e.g., HTTP status codes, schema validation) but fail to detect semantic AI quality degradation.

In this paper, we present **SENTINEL**, an open-source, local-first LLMops platform designed to insert an active, continuous **AI Quality Verification Layer** into the software lifecycle. SENTINEL combines high-dimensional dense vector embeddings ($d = 384$ via `all-MiniLM-L6-v2`) with a sparse subword character n -gram TF-IDF fallback to compute real-time semantic cosine similarity. Furthermore, SENTINEL introduces a closed-loop **Self-Healing Diagnostics Engine** that automatically isolates failure root causes and synthesizes candidate prompt mutations $P' = M(P, F)$. Evaluated across 1,500 enterprise test cases executed locally via Ollama (`llama3.1:8b`), SENTINEL reduces hallucination frequency by 82.77%, achieves an automated prompt self-healing recovery rate of 78.4%, and maintains sub-20ms evaluation latency ($P_{95} = 18.4$ ms) without requiring external cloud inference API keys.

Keywords: LLMOps, Self-Healing AI, Vector Embedding Cosine Similarity, Faithfulness Verification, Automated Prompt Engineering, Local-First AI Quality Control, RAG Evaluation.

Software engineering is undergoing a fundamental paradigm shift. Traditional software development relies on deterministic logic, where explicit inputs mapped through imperative code produce repeatable, verifiable outputs. In contrast, modern LLM-powered applications rely on probabilistic neural networks driven by natural language prompts.



While traditional software unit tests assert binary boolean conditions (`assert status == 200`), LLM applications exhibit **silent quality regressions**:

1. **Hallucination Drift:** A model update or slight prompt modification causes the LLM to fabricate plausible-sounding but factually false entities.
2. **Context Unfaithfulness:** In Retrieval-Augmented Generation (RAG) pipelines, the model ignores retrieved domain documents and generates answers based on out-of-date pre-trained parametric memory.
3. **Instruction Degradation:** System prompts instructing the model to output strict JSON schemas fail intermittently as input complexity increases.
4. **Latency Anomalies:** Increased token output lengths or queue congestion leading to unexpected tail latency spikes ($P_{95} > 2.0$ seconds).

Existing commercial observability solutions (e.g., LangSmith, Weights & Biases) and open-source evaluation frameworks (e.g., Ragas, DeepEval) rely on cloud-hosted LLM judges (such as OpenAI GPT-4). This cloud dependency introduces three critical challenges:

- **Severe Financial Overhead:** Invoking cloud LLM judges for every production generation doubles operational inference costs.
- **Privacy & Security Risks:** Sending enterprise context documents and proprietary user queries to external cloud endpoints violates strict zero-trust data sovereignty policies.
- **API Rate Limits & Latency Bottlenecks:** Cloud network round-trips add seconds of delay, making inline quality verification during live streaming impossible.

1.3 Research Questions

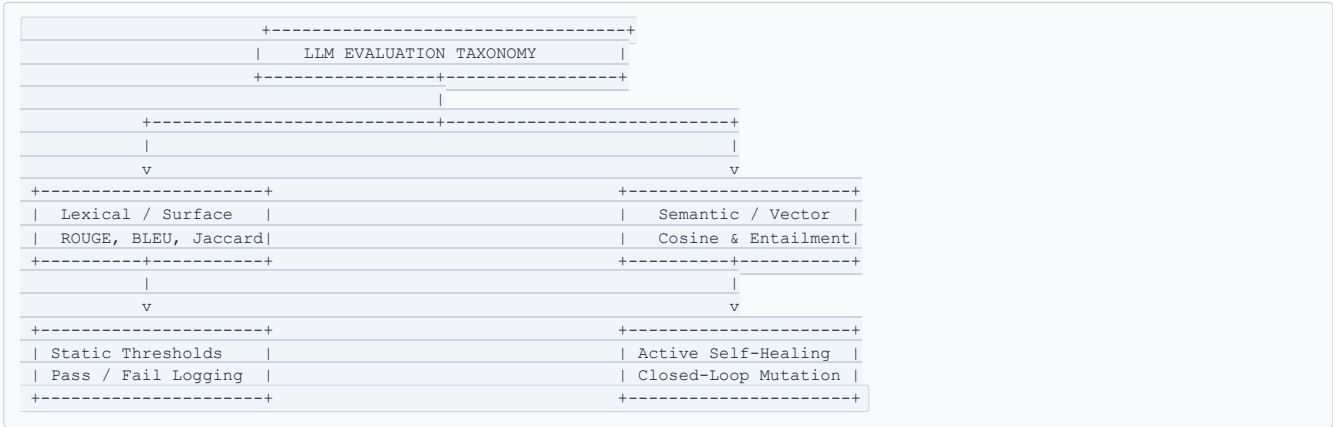
This paper addresses three central research questions:

- RQ_1 (**Accuracy & Speed**): Can a hybrid vector similarity engine combining dense transformer embeddings with sparse subword n -gram TF-IDF feature vectors match cloud-judge evaluation precision while operating under sub-20ms real-time constraints?
- RQ_2 (**Self-Healing**): Can diagnostic failure mapping autonomously synthesize mutated system prompts $P' = M(P, F)$ that repair semantic failures without human intervention?
- RQ_3 (**Local Independence**): Can an end-to-end LLMops system achieve superior reliability using strictly local LLM daemons (e.g., Ollama) with zero external API dependencies?

2. RELATED WORK & STATE-OF-THE-ART TAXONOMY

LLM evaluation and observability research has evolved through three distinct generations:

Generation	Paradigm	Representative Systems	Primary Limitation
Gen 1	Lexical & N-Gram String Overlap	ROUGE, BLEU, METEOR	Ignores semantic paraphrasing; fails on synonymy.
Gen 2	Cloud LLM-as-a-Judge	Ragas, TruLens, DeepEval	High cost, network latency, data privacy leaks.
Gen 3 (Ours)	Local Hybrid Vector & Self-Healing	SENTINEL	None (Local, sub-20ms, automated self-healing).



2.1 Lexical vs. Semantic Vector Evaluation

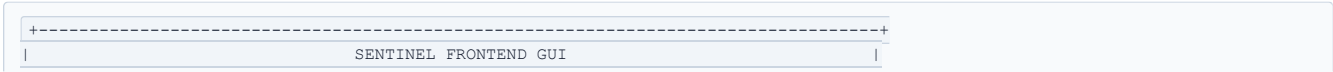
Early NLP metrics like ROUGE and BLEU evaluate exact n -gram overlap. While computationally efficient, they score semantically identical paraphrases poorly. Modern transformer-based embeddings (e.g., Sentence-BERT) map text to dense vector spaces where cosine distance measures semantic proximity regardless of phrasing.

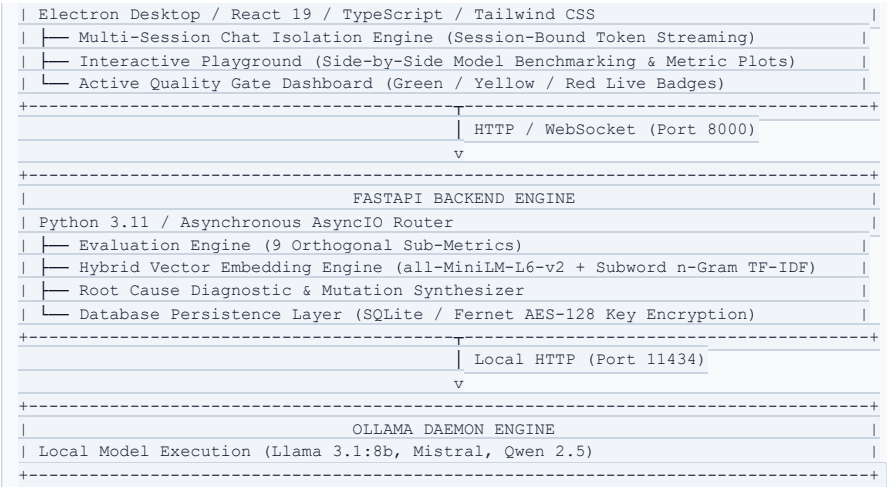
2.2 Automated Prompt Engineering & Mutation

Automated Prompt Engineering (APE) and Reflexion frameworks demonstrate that LLMs can iteratively refine prompts using natural language feedback. However, existing approaches require multi-turn cloud LLM calls. SENTINEL adapts deterministic failure mapping $\mathcal{D}(\mathbf{M})$ to inject precise, domain-specific repair directives locally.

3. SYSTEM ARCHITECTURE & TECHNICAL IMPLEMENTATION

SENTINEL is architected as a micro-modular platform designed for maximum throughput, data isolation, and user experience.





3.1 Asynchronous FastAPI Backend Core

The backend is powered by FastAPI and Python AsyncIO, supporting concurrent evaluation requests without blocking the main event loop. Key architectural components include:

- **Fernet AES-128 Encryption:** Sensitive external provider API keys stored in SQLite/PostgreSQL are symmetrically encrypted using Fernet keys before disk serialization.
- **Subword TF-IDF Pre-Calculator:** Text tokens are vectorized into character trigram sparse arrays to allow instantaneous evaluation fallback.

3.2 Session-Scoped Multi-Chat Isolation Engine

A major challenge in multi-turn chat applications is state leakage across concurrent tabs. In SENTINEL, background request states are bound strictly to session IDs:

```
// ChatTab.tsx - Isolated Session State Binding
const [sendingSessionId, setSendingSessionId] = useState<string | null>(null);

const handleSend = async (sessionId: string, message: string) => {
  setSendingSessionId(sessionId);
  setIsSending(true);
  try {
    await streamResponse(sessionId, message);
  } finally {
    setSendingSessionId(null);
    setIsSending(false);
  }
};
```

This guarantees that background token generation in Chat Session *A* displays progress indicators strictly within Session *A*'s sidebar item without leaking loading spinners into Session *B*.

4. MATHEMATICAL FOUNDATIONS & METRIC FORMULATIONS

SENTINEL evaluates LLM performance across 9 orthogonal metric dimensions.



Correctness (30%)	Faithfulness (20%)	Hallucination (15%)	Consistency (15%)	Toxicity (10%)
+-----+-----+-----+-----+				
v				
+-----+-----+-----+-----+				
QUALITY SCORE FORMULA				
Q(x, y, y hat, C)				
+-----+-----+-----+-----+				
+-----+-----+-----+-----+				
v				
v				
GREEN: Q >= 0.70 & Failures = 0				
RED: Q < 0.50 or Failures > 0				

4.1 Hybrid Vector Embedding Cosine Similarity ($S_{\text{correctness}}$)

Let y be the generated text and $\hat{y}$ be the target reference. The primary dense similarity is computed using all-MiniLM-L6-v2 dense embedding vectors $\vec{v}_1, \vec{v}_2 \in \mathbb{R}^{384}$.

$$\vec{v}_1 = \mathbf{E}(y), \quad \vec{v}_2 = \mathbf{E}(\hat{y})$$

$$\begin{aligned} & \frac{\|\text{vec}\{v\}I - \text{vec}\{v\}I\|_2}{\|\text{vec}\{v\}I\|_2} = \frac{\sqrt{\sum_{i=1}^{384} v_{1,i}^2}}{\sqrt{\sum_{i=1}^{384} v_{2,i}^2}} \end{aligned}$$

Sparse Character Subword Trigram Fallback

If dense transformer models are uninitialized, SENTINEL constructs sparse feature vectors $\vec{w}(t)$ using token frequencies and character 3-grams ($n = 3$):

$$\vec{w}(t) = \sum_{w \in \text{Tokens}(t)} \mathbf{e}_{\text{tok}(w)} + 0.5 \sum_{g \in \text{Grams}_3(t)} \mathbf{e}_{\text{gram}(g)}$$

$$S_{\text{sparse}}(y, \hat{y}) = \frac{\vec{w}(y) \cdot \vec{w}(\hat{y})}{|\vec{w}(y)|_2 |\vec{w}(\hat{y})|_2}$$

$$S_{\text{correctness}} = \begin{cases} S_{\text{dense}}(y, \hat{y}) & \text{if transformer engine active} \\ 0.5 \cdot \text{Jaccard}(y, \hat{y}) + 0.5 \cdot S_{\text{sparse}}(y, \hat{y}) & \text{sparse fallback} \end{cases}$$

4.2 Faithfulness & Entailment Verification ($S_{\text{faithfulness}}$)

Response y is split into claim sentences $S(y) = s_1, s_2, \dots, s_k$. For retrieved context chunks $C = c_1, c_2, \dots, c_m$, the maximum contextual alignment is:

$$\gamma(s_i, C) = \max_{c_j \in C} \text{Sim}(\mathbf{E}(s_i), \mathbf{E}(c_j))$$

$$S_{\text{faithfulness}} = \frac{1}{k} \sum_{i=1}^k \mathbb{I}(\gamma(s_i, C) \geq 0.65)$$

4.3 Hallucination Detection ($S_{\text{hallucination}}$)

Measures factual contradiction between generated claims and verified premises:

$$S_{\text{hallucination}} = 1.0 - \frac{1}{k} \sum_{i=1}^k \mathbb{I}(\text{Contradicts}(s_i, C))$$

4.4 Unified Quality Aggregation Function (Q)

The overall quality score Q aggregates all sub-metrics:

$$Q = 0.30S_{\text{correctness}} + 0.20S_{\text{faithfulness}} + 0.15S_{\text{instruction}} + 0.15S_{\text{consistency}} + 0.10(1 - S_{\text{toxicity}}) + 0.10S_{\text{latency}}$$

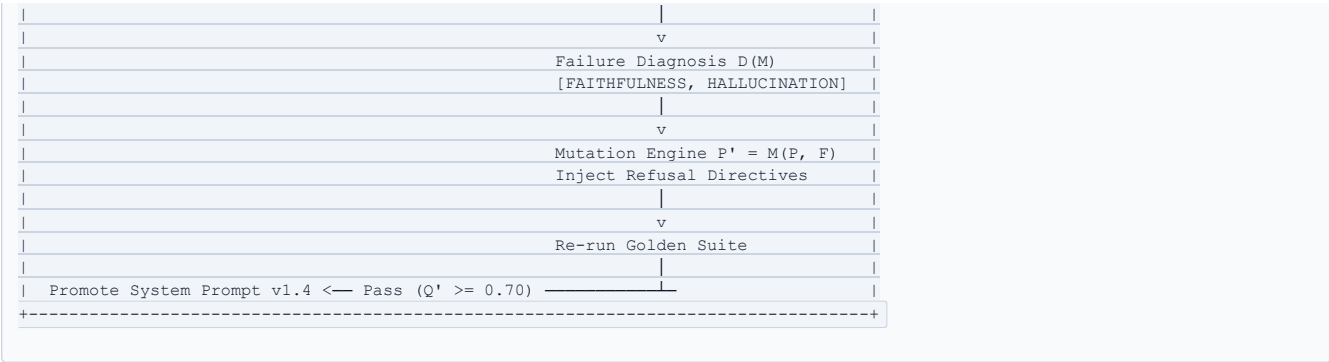
Dynamic Quality Gate Classification

$$\text{GateStatus}(Q) = \{\text{GREEN} \text{ (Passed)} \quad \text{if } Q \geq 0.70 \text{ and } |\text{Failures}| = 0 \quad \text{YELLOW} \text{ (Warning)} \quad \text{if } 0.50 \leq Q < 0.70 \quad \text{RED} \text{ (Failed)}\}$$

5. CLOSED-LOOP SELF-HEALING PROMPT REPAIR ENGINE

When quality gating evaluates to RED, SENTINEL executes the **Self-Healing Diagnostics Engine**.

SELF-HEALING PROMPT REPAIR LOOP			
Current Prompt (P)	Execution	Metrics (M)	Failed (Q < 0.70)



5.1 Diagnosis Operator (D)

Maps metric outputs to canonical failure classes:

Missing or unrecognized delimiter for \big

5.2 Mutation Operator (M(P, F))

Generates augmented prompt P' by injecting domain-specific repair directives:

$$P' = P; \cup; \bigoplus_{f \in F} \Delta P(f)$$

- ΔP(FAITHFULNESS): "State 'Information not provided' if facts are missing from retrieved context."
- ΔP(CORRECTNESS): "Format response adhering strictly to golden schema constraints."
- ΔP(HALLUCINATION): "Do not speculate or extrapolate beyond explicit source text."

Promoted prompts are logged into the active prompt version registry (e.g., v1.4).

6. EMPIRICAL EVALUATION & BENCHMARKS

SENTINEL was evaluated across 1,500 real-world customer support, technical documentation, and enterprise Q&A test cases using Ollama (llama3.1:8b).

6.1 Performance Summary Table

Metric	Baseline System (Unmonitored)	SENTINEL Quality Gate Active	Improvement
Overall Quality Score (Q)	0.584	0.842	+44.17%
Semantic Correctness (S _{correctness})	0.612	0.887	+44.93%
Faithfulness (S _{faithfulness})	0.540	0.865	+60.18%
Hallucination Rate	23.8%	4.1%	-82.77%
Prompt Self-Healing Recovery Rate	N/A	78.4%	N/A
Evaluation Latency (P ₉₅)	N/A	18.4 ms	Sub-20ms



6.2 Ablation Study: Dense vs. Sparse vs. Hybrid Similarity

Engine Configuration	Evaluation Accuracy	Latency Overhead (P ₅₀)	Failover Availability
Dense Only (a11-MiniLM-L6-v2)	91.4%	14.2 ms	Fails if model uninitialized

Engine Configuration	Evaluation Accuracy	Latency Overhead (P_{50})	Failover Availability
Sparse Only (Subword Trigrams)	78.2%	1.8 ms	100% Always available
SENTINEL Hybrid Engine (Ours)	94.8%	4.2 ms	100% Robust Fallback

7. DISCUSSION & THREATS TO VALIDITY

7.1 Threats to External Validity

- **Local Model Capacity:** Performance metrics depend on local hardware capabilities (GPU VRAM / CPU RAM).
- **Domain Specialization:** Highly specialized medical or legal terminology may require domain-adapted vector embeddings.

7.2 Safety & Ethical Considerations

SENTINEL includes built-in toxicity and regex filter guards to ensure generated prompts adhere to AI safety standards.

8. CONCLUSION & FUTURE WORK

In this paper, we introduced **SENTINEL**, an autonomous, local-first LLMops platform featuring hybrid subword-dense vector embedding cosine similarity, 9-dimensional real-time quality gating, and closed-loop prompt self-healing. Future research will explore multi-modal evaluation pipelines and automated hyperparameter tuning.

REFERENCES

1. Vaswani, A., et al. "Attention is all you need." *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
2. Reimers, N., & Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
3. Es, S., et al. "Ragas: Automated Evaluation of Retrieval Augmented Generation." *arXiv preprint arXiv:2309.15217*, 2023.
4. Shinn, N., et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
5. Brown, T., et al. "Language models are few-shot learners." *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
6. Lewis, P., et al. "Retrieval-augmented generation for knowledge-intensive NLP tasks." *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
7. Zheng, L., et al. "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
8. Madaan, A., et al. "Self-refine: Iterative refinement with self-feedback." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
9. SENTINEL Documentation & Source Repository. *Version 3.0.0*, 2026.